Classification of Titanic Passenger Survival Using Machine Learning

1. Introduction

This project focuses on predicting passenger survival on the Titanic using supervised machine learning classification techniques. The dataset, sourced from the Kaggle Titanic competition, contains demographic and travel information such as age, gender, ticket class, and fare. The objective is to build predictive models that classify whether a passenger survived or not.

This problem is a binary classification task, where the target variable is "Survived" (0 = No, 1 = Yes). Machine learning models are trained to learn patterns in the data and generalize predictions for unseen instances.

2. Data Preprocessing and Exploratory Data Analysis

Data Preprocessing

The dataset required several preprocessing steps:

- Missing values in the "Age" column were replaced with the median value.
- Missing values in "Embarked" were filled with the mode.
- Irrelevant features such as Name, Ticket, and Cabin were removed.
- Categorical variables (Sex, Embarked) were converted into numerical format using one-hot encoding.
- The dataset was split into training (80%) and testing (20%) subsets.

Exploratory Data Analysis (EDA)

EDA was conducted to understand feature relationships and distributions:

- The dataset shows class imbalance, with fewer survivors than non-survivors.
- Gender plays a significant role: females had a much higher survival rate.
- Passenger class (Pclass) strongly correlates with survival; higher-class passengers had better survival chances.
- A correlation heatmap showed that Sex and Pclass are among the most influential features.

These insights guided model selection and highlighted the importance of non-linear relationships.

3. Model Training and Evaluation

Two classification models were implemented:

Logistic Regression

A linear model used as a baseline. It assumes a linear relationship between input features and the log-odds of the target.

Random Forest Classifier

An ensemble learning method that builds multiple decision trees and aggregates their predictions. It handles non-linear relationships and feature interactions effectively.

Evaluation Metrics

Models were evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Results

The Random Forest model outperformed Logistic Regression across most metrics. It achieved higher accuracy and better balance between precision and recall. Logistic Regression performed adequately but struggled with complex feature interactions.

4. Model Interpretation

The superior performance of Random Forest is attributed to its ability to:

- Capture non-linear relationships
- Handle feature interactions
- Reduce overfitting through ensemble averaging

Feature importance analysis showed that:

- Sex (male/female)
- Fare
- Passenger class (Pclass)
  were the most influential predictors.

The confusion matrix revealed that the model correctly classified most passengers but had some difficulty predicting minority cases.

5. Deployment and Monitoring

Deployment Strategy

The trained model can be deployed using:

- A REST API built with Flask or FastAPI
- Cloud platforms such as AWS or Render
- Serialization using joblib for loading the model in production

Potential Challenges

- Data drift: incoming data may differ from training data
- Missing or inconsistent real-time inputs
- Latency in real-time predictions

Monitoring and Maintenance

To ensure long-term performance:

- Monitor prediction accuracy over time
- Log predictions and user inputs
- Retrain the model periodically with new data
- Implement validation checks for incoming data

6. Conclusion

This project demonstrated the effectiveness of machine learning in solving classification problems. The Random Forest model achieved the best performance due to its flexibility and robustness. Proper preprocessing, EDA, and evaluation were essential in building a reliable model. Deployment and monitoring strategies ensure the model remains effective in real-world applications.